

For the project, we hold out a portion of data to test your model. We need a way to store and execute your models, but neither GitHub nor Gradescope accommodates student files over 100 MB. The process described here provides flexibility, but makes submitting a little non-trivial.

There are two intermediaries in this process: UV Package Manager and HuggingFace (HF). You store your preprocessing pipeline and fitted model in a file, define versions and dependencies, and upload all that to Gradescope. Instead of Gradescope, you will submit a public HF repository name that our autograders will pull your model.

Both *serializing* (aka storing) your model and *deserializing* (aka loading) require the same versions of Python and your dependencies (e.g., pandas). Exact replication of your environment and dependencies requires two basic files: **pyproject.toml** to specify dependencies and project metadata and **.python-version** to specify the Python version for the environment. **It** also permits you to use external libraries as you please.

Hugging Face: [Hugging Face](https://huggingface.co/welcome) (HF) is a leading open-source AI platform, a kind of GitHub for ML, where you can **store large models** hasslefree. For now, all you need is a HuggingFace account. Please sign up for one here: <https://huggingface.co/welcome>

UV Package Manager: UV is an industry-standard Python package manager used to ensure an exact match between your local development environment (e.g., your laptop) and the grading environment (which will not be Gradescope). To install UV on your computer, run ``pip install uv`` or use any of the alternative methods found in the [UV Installation Docs](#).

Here are 3 essential UV commands:

- ``uv add <package>``: Adds a library to the project and updates ``pyproject.toml``.
- ``uv sync``: Synchronizes the project environment with the ``uv.lock`` file by either creating or updating a virtual environment (i.e., installs/removes/updates dependencies).

Reference Code

As you view the following instructions, reference the following files in the [Model Submission Template GitHub classroom assignment](#).

- Sample notebooks **demo.ipynb** and **complex_demo.ipynb** to see how the process is implemented in practice. The only difference between them is the dataset on which the model is trained and preprocessing methods. The core process remains the same.
- **submit.py**: Contains the utility function **upload_files()** that you must call to upload your trained models to HuggingFace
- **pyproject.toml**: Contains base dependencies (e.g., cloudpickle, scikit-learn, pandas) that are needed to run your notebook/serialize your model
- **.python-version**: Defines the python version used when running your notebook locally
- **docker-compose.yaml** (**optional**): Defines the docker services for running the local autograders for each of the notebooks provided (see instructions below). Note that the **iris-runner** service is used for running the autograder for models trained in **demo.ipynb** and the **complex-runner** is used for running that of **complex_demo.ipynb**!

General Instructions

1. If using Google Colab, download your notebook to your local machine.
2. Ensure your local environment can run the notebook:
 - a. Add all missing dependencies into the UV project using ``uv add``
Do not change the versions of the existing modules (e.g., cloudpickle) provided in the template's pyproject.toml
 - b. Modify the `.python-version` file if necessary to match your Python version
4. Run **uv sync** to create a new virtual environment with those dependencies and version
 - a. Open your notebook locally and make sure it's using the Python interpreter/kernel of the virtual environment you just created. This will vary by IDE.
5. Now wrap both your preprocessing and estimator stages (e.g., ColumnTransformers, predictor/estimator, etc.) into a single pipeline. (For details on Scikit-learn pipelines and custom transformers, see the [documentation](#) and reference their attached notebooks.)

In essence, the pipeline you make must accept the original data (without pre-processing) as an argument to its ``predict()`` method without breaking.

The holdout data maintains the same schema as the data we provide to you. If it doesn't work on that, it will definitely not work on Gradescope!

6. Ensure the pipeline is fitted and ready for prediction. Do either a or b:
 - a. Re-fit this pipeline to your training data
 - b. Add your already fitted pipelines into a new pipeline as-is
7. Save the fitted model to ``model.pkl`` using ``cloudpickle`` as seen in the demo notebooks.
8. Call the `login()` in your notebook method from the `huggingface_hub` module.
 - a. Make sure you create/select a key with ``write`` permissions (that is, allows you to write/upload objects to your HF repos)
 - b. Create a `HfApi` object
9. Upload your files to a HuggingFace repository by calling the ``upload_files`` method provided in the ``submit`` python module we've given, passing in the ``HfApi`` object you instantiated in the previous step. Running this will:
 - Upload **model.pkl**, **pyproject.toml** and **.python-version** to the repo specified by the argument, creating the repo if it doesn't already exist.
 - Save a **repo.json** object to your local directory containing the following schema:
`{ 'repo_id' : <repo_you_uploaded_to> }`
10. Upload the ``repo.json`` object to Gradescope or optionally/if you have any errors, pass to your local Docker image (see below)

Debugging and testing with Docker (optional)

If you encounter problems, you may choose to use Docker and Docker Compose to help you debug. Think of it this way: the environment on Gradescope is obviously different from all of our individual machines. For example, most of us aren't using Linux; Gradescope is. That means that if your code runs locally on your machine, it's not guaranteed to work on Gradescope. So what if we could replicate the Gradescope environment completely and run our code inside that environment? Wouldn't that be effectively the same as running it on Gradescope yourself? **Well, that “replicated environment” is a container and that is what we'll be doing.**

Docker is a platform that wraps your software into a standardized unit called a **container**. This container has everything the software needs to run: code, runtime, system tools, and libraries. We will distribute a container for you to test your submissions on, the only difference is that your containers will use the data we've already given you and let you view logs, while our containers will use our own secret holdout set and hide the logs from you to prevent data leakage.

Docker lets you simulate running the autograder locally so you know what's happening during the execution process and debug as necessary if there are errors. For example, if your pipeline doesn't match our specifications, submitting to Gradescope would say that it just didn't work. Running with Docker will show error messages that can help you discern *why*. To install Docker, follow the Docker/Docker Compose Docs here <https://docs.docker.com/compose/install>.

Docker Compose is an extension of Docker that makes running containers easier by providing easier configuration via docker-compose files. This allows you to replicate the Linux environment used by Gradescope on your local machine for debugging.

If you're having trouble running on Gradescope or would like to test that you did things correctly:

1. Open Docker Desktop. **Wait for the bottom bar to say “Docker engine running”**
2. Navigate to the same directory as the given **docker-compose.yaml** file
3. If not already present, create two folders with the following names:
 - a. **submission**
 - b. **results**
4. Copy your **repo.json** into the **submission** folder.
5. Open a terminal and run ``docker compose run <service_name>`` in the **same directory** as the **docker-compose.yaml** file.
 - a. Note that the service name is found in the docker compose file as the first indentation beneath the ``services`` attribute
 - b. As a convention, we'll always name them to end in **“runner”** (eg. iris-runner)
6. Verify that your code runs!

If you need to inspect any errors/failures:

7. Explore the ``/results/results.json`` for details
8. Open Docker Desktop and inspect the logs of the container that just ran (though they should already be in the terminal you are running)